Spec-Driven Development

A Standard for Structured Software Development
with AI Agents

Version 0.1 — March 2026

Table of Contents

Introduction

- Spec-Driven Development (SDD) Standard

The Standard

- Principles
- Workflow
- Roles
- Anatomy of a Spec
- Boundaries
- Tool Agnosticism
- Glossary
- Adoption Guide

Guides

- Getting Started
- PO Guide to SDD
- From Story to Spec
- Writing Requirements
- Writing Design Docs
- Breaking Down Tasks
- Spec Review Checklist
- Brownfield Adoption
- Greenfield Setup

Templates

- Product Brief Template
- Steering Template
- Feature Spec Template
- Bugfix Spec Template
- Standards Template
- Task List Template

Examples

- Examples
- Notification Preferences — Product Brief
- Notification Preferences — Requirements
- Notification Preferences — Design
- Notification Preferences — Tasks
- Duplicate Emails on Order Confirmation — Bugfix Analysis
- Duplicate Emails on Order Confirmation — Fix Tasks
- TaskFlow

Spec-Driven Development (SDD) Standard

Version 0.1 | March 2026

Spec-Driven Development is a methodology where structured specifications are the primary artifact guiding software development. Humans define the *what* and *why* through specs; AI agents and developers handle the *how* within clear boundaries. The result: predictable, reviewable, high-quality software — whether built by humans, AI, or both.

SDD bridges the gap between product vision and implementation. It gives Product Owners a structured way to express intent that both developers and AI agents can act on, and gives developers a clear contract to design and build against.

The Standard

- Principles — The core beliefs that drive SDD
- Workflow — The 6-phase lifecycle from brief to validation
- Roles — PO/PM, Developer, and AI Agent responsibilities
- Anatomy of a Spec — What makes a good spec
- Boundaries — The Always/Ask/Never framework
- Glossary — Precise definitions of SDD terms

Templates

Copy these into your project to get started:

- Product Brief Template — PO/PM starting point before spec creation
- Steering Template — Project constitution (CLAUDE.md / .cursorrules)
- Feature Spec Template — New feature specifications
- Bugfix Spec Template — Bug investigation and fix specifications
- Standards Template — Project-wide coding conventions
- Task List Template — Implementation task breakdowns

Guides

- Getting Started — 5-minute minimal setup
- PO Guide to SDD — How Product Owners write specs for dev teams and AI
- From Story to Spec — Translating product vision into SDD artifacts
- Writing Requirements — Requirements notation guide (multi-format)
- Writing Design Docs — Design document guide
- Breaking Down Tasks — Task decomposition guide
- Spec Review Checklist — Validation checklist
- Brownfield Adoption — Adopting SDD in an existing codebase
- Greenfield Setup — Starting a new project with SDD

Examples

Each example uses the exact folder and file structure prescribed by the standard. See Examples for the full overview.

- **Feature Spec:** `Examples/notification-preferences/` — brief → requirements → design → tasks
- **Bugfix Spec:** `Examples/duplicate-order-emails/` — bugfix → tasks
- **Steering Doc:** `Examples/taskflow-steering/` — CLAUDE.md

Adoption

- Adoption Guide — How to adopt SDD incrementally
- Tool Agnosticism — How SDD works across Claude, Cursor, Copilot, Kiro, and more

Part: The Standard

Principles, workflow, roles, and definitions

Principles

The core beliefs that drive Spec-Driven Development. These principles are not rules to follow mechanically — they are the reasoning behind every practice in the standard.

---

1. Specs Before Code

Time invested in specification reduces total implementation time. A clear spec eliminates ambiguity upfront, preventing costly rework. This is especially true with AI agents: a precise spec produces correct code on the first pass; a vague prompt produces code that needs multiple rounds of correction.

2. Feature-Scoped, Not Monolithic

One spec folder per feature or bugfix. Never a single giant specification document covering the entire project. Small, focused specs are easier to write, review, implement, and validate. They enable parallel work and clear ownership.

3. The Constitution Is Not a Spec

The steering document (e.g., `CLAUDE.md`) defines project identity — tech stack, conventions, boundaries. Feature specs define what to build. These are distinct concerns:

- **Steering document**: "We use TypeScript, follow these naming conventions, never modify production configs."
- **Feature spec**: "Build a password reset flow that sends an email with a time-limited token."

Conflating them leads to bloated context files that are neither good constitutions nor good specs.

4. Smarter Specs, Not Longer Specs

The "curse of instructions": as you add more instructions, adherence to each one drops. A 50-line spec with precise, testable requirements outperforms a 500-line spec full of vague guidance. Every sentence in a spec should earn its place.

5. Portable Across Tools

Specs are Markdown files. They work with Claude Code, Cursor, Copilot, Kiro, Windsurf, or any future tool that reads files. The standard never depends on a specific vendor's features. Tool-specific integration is an add-on, not a requirement.

6. Incremental Adoption

You don't need to adopt the full standard on day one. Start with a steering document. Add feature specs when complexity warrants it. Introduce the Product Brief when PO/PM collaboration becomes important. The standard is designed to be adopted in layers. See Adoption Guide.

7. Living Documents

Specs can and should be updated during implementation. When building reveals something the design didn't anticipate, update the spec — don't silently diverge. The spec is the source of truth; if reality doesn't match the spec, one of them must change.

8. Human Decides, AI Executes

The human (PO/PM or Developer) owns every decision. The AI agent is a powerful executor, not an autonomous decision-maker. When the AI encounters ambiguity, the correct behavior is to flag it — not to guess.

9. Validate Against the Spec

Implementation is checked against the spec, not against intuition. Acceptance criteria from requirements become test cases. If it's not in the spec, it's not a requirement — and if it is in the spec, it must be verified.

10. Product Intent Drives Specs

The PO/PM owns the *why* and the *what*. Specs are the bridge between product vision and implementation. A well-written spec serves two audiences simultaneously: human developers who need context and rationale, and AI agents who need precision and explicit boundaries. See Roles.

Workflow

The SDD lifecycle has six phases, from product intent to validated implementation. Each phase has a clear owner, defined inputs and outputs, and a review gate before proceeding.

Overview

```
Brief → Specify → Design → Task → Implement → Validate
 (PO)   (PO+Dev)  (Dev)   (Dev+AI)  (AI+Dev)   (PO+Dev)
```

Two entry points are supported:

- **Requirements-First** (default): Start at Phase 0 (Brief). PO-driven. Best for product-led development.
- **Design-First**: Start at Phase 2 (Design), then derive requirements. Dev-driven. Best when architecture is the primary constraint (porting, compliance, technical migration).

Phase 0: Brief

Owner: PO/PM

Input: A product idea, user problem, or business need

Output: `brief.md`

The PO writes a Product Brief capturing:

- The problem being solved and who it affects
- What success looks like (measurable where possible)
- What's in scope and what's explicitly out
- Known constraints and open questions

The brief is intentionally non-technical. It captures *intent*, not implementation. It's the starting point — not the final spec.

Review gate: Dev reviews the brief for feasibility and asks clarifying questions before proceeding.

Phase 1: Specify

Owner: PO/PM (content) + Developer (feasibility review)

Input: `brief.md`

Output: `requirements.md`

Transform the brief into structured requirements with acceptance criteria. The PO writes the *what*; the Developer validates that each requirement is technically feasible.

Requirements can use any supported notation — see Writing Requirements:

- **User stories** for capturing user intent and empathy
- **EARS notation** for precise system behavior
- **Acceptance criteria (Given/When/Then)** for testable conditions
- **Freeform requirements** for straightforward needs

Every requirement must have at least one acceptance criterion. If you can't write a test for it, the requirement is too vague.

Review gate: PO and Dev agree that requirements are complete, feasible, and testable.

Phase 2: Design

Owner: Developer

Input: `requirements.md`

Output: `design.md`

The Developer writes the technical design covering:

- Architecture decisions and component interactions
- Data models and state management
- API contracts and integration points
- Error handling strategy
- Testing approach

The design must trace back to requirements — every design decision should serve at least one requirement. Design decisions without a corresponding requirement indicate either a missing requirement or unnecessary complexity.

Review gate: PO reviews for alignment with original intent. Dev validates technical soundness.

Phase 3: Task

Owner: Developer + AI Agent (assistant)

Input: `design.md`

Output: `tasks.md`

Break the design into an ordered list of atomic implementation tasks. Each task should be:

- **One logical change** — typically one commit
- **Independently testable** — you can verify it works without completing later tasks
- **Sequenced by dependency** — tasks that depend on earlier work come later
- **Clear in scope** — the task description leaves no room for interpretation

The AI agent can help decompose the design into tasks, but the Developer validates the breakdown. See Breaking Down Tasks.

Each task in `tasks.md` tracks status: `[ ]` not started, `[-]` in progress, `[x]` complete.

Review gate: Dev confirms the task list fully covers the design.

Phase 4: Implement

Owner: AI Agent + Developer (reviewer)

Input: `tasks.md` + `design.md` + `requirements.md` + steering document

Output: Working code

The AI agent reads the full spec context and executes tasks in order. The Developer reviews output after each task (or batch of tasks, depending on trust level and complexity).

During implementation:

- The spec is the primary instruction set for the AI agent
- If the agent encounters ambiguity, it flags it rather than guessing

- If implementation reveals a design issue, **update the spec** — don't silently diverge
- Boundaries define what the agent must always do, must ask about, and must never do

---

Phase 5: Validate

Owner: PO/PM + Developer

Input: Completed implementation + `requirements.md`

Output: Validated feature or identified gaps

Walk through each requirement and verify it is met:

- **PO/PM validates user value**: Does the implementation solve the original problem? Do acceptance criteria pass?
- **Developer validates technical quality**: Does the code meet standards? Are there regressions?

If validation reveals gaps:

- Update the spec to reflect what was learned
- Create new tasks for remaining work
- Re-enter the workflow at the appropriate phase

The Portable Spec Structure

Any project adopting SDD uses this folder structure:

```
project-root/
  CLAUDE.md (or .cursorrules, etc.)   # Steering document
  specs/
    [feature-name]/
      brief.md                        # (optional) Product Brief — PO/PM starting point
      requirements.md                 # Structured requirements + acceptance criteria
      design.md                       # Architecture & design decisions
      tasks.md                        # Ordered atomic tasks
      research.md                     # (optional) Alternatives explored
    [bugfix-name]/
      bugfix.md                       # Current vs expected vs unchanged behavior
      tasks.md                        # Fix implementation tasks
  standards/                          # (optional) Project-wide coding conventions
    [topic].md
```

Naming conventions:

- Spec folders use hyphenated slugs: `user-authentication/`, `payment-processing/`
- `specs/` is visible and version-controlled, not hidden (no `.specs/` or `.kiro/`)
- The steering document filename is tool-specific (`CLAUDE.md`, `.cursorrules`, etc.) but follows the SDD content structure

---

When to Use SDD

SDD is for work that benefits from structured planning:

- Features requiring multiple components or integration points
- Bugs where regressions are costly
- Work involving PO/Dev/AI collaboration
- Any task where "just start coding" leads to rework

For quick fixes, typo corrections, or exploratory prototyping — skip the ceremony and just do it. You can always generate a spec retroactively if the work grows in scope.

Roles

SDD defines three actors with distinct responsibilities. Clear ownership prevents gaps (nobody owns it) and conflicts (everybody owns it).

Product Owner / Product Manager

The PO/PM is the voice of the user and the business. They own the *why* and the *what*.

Responsibilities:

- Write the Product Brief — the starting artifact that captures problem, need, and success metrics
- Author and approve requirements in `requirements.md`
- Define acceptance criteria that are testable by both humans and AI agents
- Review design documents for alignment with product intent (not technical correctness)
- Validate the final implementation against requirements — does it solve the original problem?

Key skill in SDD: Writing for two audiences simultaneously. Human developers need context, narrative, and rationale. AI agents need precision, explicit boundaries, and testable criteria. A good SDD requirement serves both.

Phase ownership:

Phase	PO/PM Role
Brief	Owns — writes the Product Brief
Specify	Owns — writes requirements; Dev reviews feasibility
Design	Reviews — checks alignment with intent
Task	Informed — may clarify scope questions
Implement	Informed — available for clarification
Validate	Owns — validates user value and acceptance criteria

Common anti-patterns to avoid:

- Vague acceptance criteria ("it should be fast" → "page loads in under 2 seconds")
- Assumed context ("like we did last time" → spell it out; AI agents have no memory of last time)
- Scope creep via ambiguity ("and maybe also..." → define explicit scope boundaries)
- Over-specifying the *how* (leave technical decisions to the Developer)

Developer

The Developer owns the *how*. They translate requirements into architecture and code.

Responsibilities:

- Review the Product Brief and requirements for technical feasibility
- Write the design document (`design.md`) — architecture, components, data models, API contracts
- Break the design into atomic tasks (`tasks.md`)
- Review AI-generated code for quality, correctness, and adherence to standards
- Maintain the steering document with project-wide conventions
- Validate technical quality during the Validate phase

Phase ownership:

Phase	Developer Role
Brief	Reviews — flags feasibility concerns
Specify	Reviews — validates feasibility of each requirement
Design	Owns — writes design.md
Task	Owns — writes tasks.md (AI may assist)
Implement	Reviews — validates AI output per task
Validate	Owns — validates technical quality

AI Agent

The AI agent is a powerful executor that works within boundaries. It does not make product or architectural decisions.

Responsibilities:

- Read the full spec context (steering document + spec folder) before starting work
- Execute tasks in the order defined in `tasks.md`
- Follow the Always/Ask/Never framework strictly
- Flag ambiguity or contradictions rather than guessing
- Produce code that is reviewable in small, task-sized increments

What the AI agent should never do:

- Make product decisions (that's the PO/PM)

- Make architectural decisions not covered by the design (that's the Developer)
- Skip tasks or change task order without human approval
- Modify files outside the scope defined in the spec
- Assume context that isn't explicitly stated in the spec or steering document

The spec is the agent's primary instruction set. The steering document provides project-wide rules; the spec folder provides feature-specific context. Together, they give the agent everything it needs to execute correctly.

Collaboration Model

```
PO/PM                Developer              AI Agent
  │                     │                      │
  ├─ Brief ─────────────►│                      │
  │                     │                      │
  ├─ Requirements ──────►│                      │
  │  (Dev reviews)      │                      │
  │                     ├─ Design              │
  │  (PO reviews)◄──────┤                      │
  │                     │                      │
  │                     ├─ Tasks ─────────────►│
  │                     │                      │
  │                     │◄── Implementation ───┤
  │                     │   (Dev reviews)      │
  │                     │                      │
  ├─ Validation ◄───────┤                      │
  │  (PO: value)        │  (Dev: quality)      │
```

Each handoff is a review gate. No phase proceeds without the responsible party's approval.

Anatomy of a Spec

What makes a good spec? This document defines the six areas every feature spec should cover and explains what quality looks like in each.

The Six Areas

A complete feature spec addresses these areas across its documents (`brief.md`, `requirements.md`, `design.md`, `tasks.md`):

1. Objective and Purpose

Where: `brief.md` and the opening section of `requirements.md`

Owner: PO/PM

What problem does this feature solve? Why now? What does success look like?

A good objective is:

- **Specific** — names the user and the problem
- **Measurable** — includes success criteria that can be verified
- **Bounded** — states what's in scope and what's out

> *NOTE: > Weak: "Improve the login experience."*
> *Strong: "Allow users who forgot their password to reset it via email, reducing support tickets for password resets by 80%."*

2. Requirements

Where: `requirements.md`

Owner: PO/PM (content) + Developer (feasibility)

Structured statements of what the system should do. See Writing Requirements for notation options.

A good requirement is:

- **Testable** — you can write a pass/fail check for it
- **Independent** — it doesn't secretly depend on another unstated requirement

- **Unambiguous** — two people reading it would agree on what it means
- **Necessary** — removing it would leave the feature incomplete

Every requirement must have at least one acceptance criterion.

3. Design Decisions

Where: `design.md`

Owner: Developer

How will the feature be built? Architecture, component interactions, data models, API surface.

A good design:

- **Traces to requirements** — every decision serves at least one requirement
- **Records alternatives considered** — especially when the choice is non-obvious
- **Defines interfaces** — what goes in, what comes out, what errors are possible
- **Considers edge cases** — what happens when things go wrong

4. Constraints and Boundaries

Where: Spread across `requirements.md` (feature-specific) and the steering document (project-wide)

Owner: PO/PM (product constraints) + Developer (technical constraints)

What the implementation must do, must ask about, and must never do. See Boundaries for the Always/Ask/Never framework.

Constraints include:

- Performance requirements
- Security requirements
- Compatibility requirements
- Dependencies that must or must not be used
- Files or systems that must not be modified

5. Task Decomposition

Where: `tasks.md`

Owner: Developer (+ AI Agent as assistant)

The ordered sequence of atomic work items. See Breaking Down Tasks.

Good tasks are:

- **Atomic** — one logical change per task
- **Ordered** — dependencies flow forward, never backward
- **Testable** — each can be verified independently
- **Specific** — no interpretation needed; the task describes exactly what to do

6. Validation Criteria

Where: Acceptance criteria in `requirements.md`, verification steps in `tasks.md`

Owner: PO/PM (user value) + Developer (technical quality)

How do we know it's done? This is not an afterthought — it's the bridge between specifying and validating.

Good validation criteria:

- Map 1:1 to requirements
- Are written before implementation begins
- Are executable (manually or automatically)
- Cover both happy paths and error cases

The Quality Test

A spec is ready for implementation when:

- A developer who has never seen the project could read the spec and understand what to build
- An AI agent could read the spec and execute tasks without needing to ask clarifying questions
- After implementation, you could verify every requirement by checking its acceptance criteria
- There are no implicit assumptions — everything the implementer needs to know is written down

If any of these fail, the spec needs more work before proceeding to implementation.

Spec Size

Follow the smarter specs, not longer specs principle:

- A typical feature spec is **2-4 pages** across all documents combined
- If your requirements document exceeds 2 pages, consider splitting the feature into smaller specs
- If your task list exceeds 15 tasks, the feature may be too large for a single spec
- Every sentence should earn its place — if removing it wouldn't change the implementation, remove it

Boundaries

The Always/Ask/Never framework defines explicit constraints on what AI agents and developers should do during implementation. Boundaries exist at two levels: **project-wide** (in the steering document) and **feature-specific** (in the spec's requirements).

The Framework

Always

Things the AI agent must do without asking. Non-negotiable conventions.

Project-wide examples:

- Follow existing code conventions and naming patterns
- Run tests before marking a task complete
- Write code in the language specified by the steering document
- Keep changes within the scope defined by the current task

Feature-specific examples:

- Validate email format before sending reset link
- Log all authentication attempts
- Use the existing database schema — do not create new tables

Ask

Things the AI agent must confirm with a human before doing. Decisions requiring judgment.

Project-wide examples:

- Adding a new dependency or library
- Changing a public API contract
- Modifying shared configuration files
- Deviating from the design document

Feature-specific examples:

- Choosing between two valid implementation approaches
- Handling an edge case not covered in requirements

- Modifying a file outside the spec's stated scope

Never

Things the AI agent must not do. Hard prohibitions.

Project-wide examples:

- Modify environment variables or secrets
- Skip tests or disable linting
- Push directly to main/production branches
- Alter CI/CD pipeline configuration
- Commit files containing credentials

Feature-specific examples:

- Modify unrelated features or components
- Change the database schema without explicit approval
- Downgrade security measures

Where Boundaries Live

Level	File	Scope
Project-wide	Steering document (CLAUDE.md, .cursorrules,	Apply to all work in the project
Feature-specific	requirements.md (Boundaries section)	Apply only to this feature's implementation

Feature-specific boundaries **add to** project-wide boundaries — they never override them. If the steering document says "never modify production configs," a feature spec cannot grant permission to do so.

Writing Good Boundaries

Be specific, not aspirational:

- Weak: "Be careful with security"
- Strong: "Never store passwords in plaintext; always use bcrypt with a minimum cost factor of 12"

Be actionable, not philosophical:

- Weak: "Follow best practices"
- Strong: "Always use parameterized queries for database access"

Cover the edges:

- What happens when the AI encounters something not covered by the spec?
- What files or systems are explicitly off-limits?
- What decisions require human approval even if they seem small?

For PO/PMs

Product boundaries are often the most important and the most overlooked. As a PO/PM, define:

- **Data boundaries** — What user data can/cannot be accessed, stored, or modified?
- **Scope boundaries** — What is explicitly out of scope for this feature?
- **Experience boundaries** — What existing user flows must not be disrupted?
- **Compliance boundaries** — What regulatory or policy constraints apply?

These translate directly into the Always/Ask/Never framework and help AI agents stay within the intended scope.

Tool Agnosticism

SDD is designed to work with any AI coding tool. Specs are Markdown files — any tool that reads files can use them.

Core Principle

The SDD standard never depends on a specific vendor's features. Tool-specific integration is an add-on, not a requirement. If your AI tool disappears tomorrow, your specs remain valuable as documentation and can be used with any replacement.

How SDD Maps to Popular Tools

Claude Code

SDD Concept	Claude Code Equivalent
Steering document	CLAUDE.md at project root
Spec context	Read spec files directly or reference via conversation
Boundaries	Defined in CLAUDE.md and respected by Claude
Task execution	Claude reads tasks.md and executes sequentially

Cursor

SDD Concept	Cursor Equivalent
Steering document	.cursorrules at project root
Spec context	Reference spec files in chat or as context
Standards	Can be included as rules or referenced files
Task execution	Use Composer with spec context

GitHub Copilot

SDD Concept	Copilot Equivalent

Steering document	.github/copilot-instructions.md
Spec context	Reference spec files in Copilot Chat
Standards	Include in instructions file or reference separately

Kiro

SDD Concept	Kiro Equivalent
Steering document	Project-level context
Spec folder	.kiro/specs/[feature]/ (Kiro's native format)
Requirements	requirements.md (matches Kiro's native format)
Design	design.md (matches Kiro's native format)
Tasks	tasks.md (matches Kiro's native format)

SDD's spec structure is intentionally compatible with Kiro's format. Migration between SDD and Kiro is trivial.

Other Tools

Any AI tool that can read files from the project directory can use SDD specs. The minimum requirement is:

- The tool can read the steering document for project context
- The tool can read spec files when given a file path
- The tool follows instructions written in Markdown

The Steering Document Bridge

The steering document is the only file with a tool-specific name. Everything else is standard Markdown in standard directories.

If you use multiple tools on the same project:

- Use `CLAUDE.md` as the primary (most tools can read any named file)
- Symlink or copy for tools that require a specific filename
- Or maintain parallel files with the same content (less ideal, but works)

The content structure remains identical regardless of filename. See Steering Template.

Portable Spec References

When referencing specs in conversations with AI tools:

```
Read the spec at specs/password-reset/requirements.md and implement task 3
from specs/password-reset/tasks.md
```

This works identically across all tools. The spec is the context — the tool is just the execution environment.

Glossary

Precise definitions of terms used throughout the SDD Standard. When a term appears in other documents, it carries exactly the meaning defined here.

Term	Definition
Acceptance Criteria	Testable conditions that must be true for a requirement to b
AI Agent	An AI-powered coding assistant (e.g., Claude Code, Cursor, C
Boundary	An explicit constraint on what the AI agent or developer mus
Brief	See Product Brief.
Brownfield	An existing codebase where SDD is being adopted. Contrast wi
Constitution	See Steering Document.
Design Document	The design.md file within a spec folder. Captures architectu
EARS Notation	Easy Approach to Requirements Syntax. A structured format fo
Feature Spec	A set of documents (brief, requirements, design, tasks) in a
Greenfield	A new project being built from scratch with SDD. Contrast wi
Phase Gate	A review point between workflow phases where a human (PO/PM
Product Brief	The brief.md file within a spec folder. The PO/PM's starting

Product Owner (PO)

The person responsible for defining what to build and why. O

Requirements Document	The requirements.md file within a spec folder. Contains stru
Spec	Short for specification. A feature-scoped set of documents (
Spec Folder	A directory under specs/ named after the feature or bugfix i
Steering Document	The project-level constitution file (e.g., CLAUDE.md, .curso
Task	An atomic unit of implementation work defined in tasks.md. S
Task List	The tasks.md file within a spec folder. An ordered sequence
Validation	The final workflow phase where implementation is checked aga

Adoption Guide

SDD is designed for incremental adoption. You don't need to implement everything on day one — adopt layer by layer, adding rigor as your needs grow.

Adoption Levels

Level 1: Steering Only

Effort: 15 minutes

What you do: Create a steering document at your project root.

What you get: AI agents understand your project context, conventions, and boundaries. Every AI interaction improves immediately.

This alone is a significant upgrade over unstructured AI usage.

Level 2: Steering + Feature Specs

Effort: 30-60 minutes per feature

What you do: Add `specs/` directory. Write requirements, design, and tasks for complex features.

What you get: Predictable, reviewable implementation. Clear handoff between planning and execution.

Level 3: Full SDD with Product Briefs

Effort: Full workflow per feature

What you do: PO/PM writes Product Briefs. Full 6-phase Workflow with review gates.

What you get: Clear PO/Dev/AI collaboration model. Product intent preserved through implementation.

Level 4: Standards Library

Effort: Ongoing

What you do: Capture project conventions in `standards/` directory.

What you get: Consistent code across features and team members. AI agents follow project patterns automatically.

Adoption Path

```
Level 1          Level 2              Level 3              Level 4
Steering    →    + Feature Specs  →   + Product Briefs →   + Standards
(15 min)         (per feature)        (full workflow)       (ongoing)
```

Move to the next level when:

- **Level 1 → 2:** You're building features complex enough that "just start coding" leads to rework
- **Level 2 → 3:** Multiple people are involved in defining what to build (PO + Dev, or multiple stakeholders)
- **Level 3 → 4:** You notice recurring patterns that should be codified

---

Signals You Need More Structure

Signal	Suggested Action
AI agent keeps making the same mistakes	Add relevant boundaries to steering document (Level 1)
Features get built but don't match what was wanted	Start writing requirements with acceptance criteria (Level 2
PO/PM and Dev have different understanding of the feature	Introduce Product Briefs (Level 3)
Code style varies across features	Create coding standards (Level 4)
New team members take a long time to get productive	The steering document and standards reduce onboarding time

Signals You Have Too Much Structure

Signal	Suggested Action
Writing specs takes longer than implementing	Use lighter specs — not everything needs EARS notation
Team avoids specs because they're a burden	Reduce ceremony — start with just requirements + tasks, skip
Specs are always outdated after implementation	Enforce the "update the spec" practice in validation, or acc

---

Tips for Successful Adoption

- **Start with one person, one feature.** Don't mandate SDD across the team. Let the results speak.
- **Pick the right first feature.** Medium complexity, upcoming, self-contained. See Brownfield Adoption.
- **Don't spec everything.** Quick fixes, typo corrections, and exploratory work don't need specs.
- **Adapt the templates.** The templates are starting points, not rigid forms. Remove sections that don't apply.
- **Review and iterate.** After 3-5 specs, review what worked and what was unnecessary. Adjust your approach.

Part: Guides

Practical how-to guides for teams adopting SDD

Getting Started

Adopt SDD in 5 minutes. This guide gives you the minimal viable setup — you can add more structure later as your project grows.

Step 1: Create Your Steering Document (2 minutes)

Copy the Steering Template to your project root. Name it based on your AI tool:

Tool	Filename
Claude Code	CLAUDE.md
Cursor	.cursorrules
GitHub Copilot	.github/copilot-instructions.md
Multiple tools	CLAUDE.md (most tools can read any named file)

Fill in the essentials:

- Project overview (1 sentence)
- Tech stack (be specific)
- Key commands (build, test, lint)
- 2-3 boundaries per category (Always/Ask/Never)

You can skip the full Project Structure and Code Conventions sections for now — add them when you feel the need.

Step 2: Create Your Specs Folder (30 seconds)

```
mkdir specs
```

That's it. The folder is empty until you need your first spec.

Step 3: Write Your First Spec (when ready)

When you have a feature that's complex enough to benefit from planning:

- Create a folder: `specs/[feature-name]/`
- **(PO/PM)** Write a `brief.md` using the Product Brief Template — or skip this and go directly to requirements
- **(PO/PM + Dev)** Write `requirements.md` using the Feature Spec Template
- **(Dev)** Write `design.md`
- **(Dev)** Write `tasks.md`
- **(AI + Dev)** Execute tasks
- **(PO/PM + Dev)** Validate against requirements

What You Can Skip

SDD is designed for incremental adoption. Here's what's optional:

Component	When to add it
brief.md	When PO/PM and Dev are different people, or when product con
standards/ folder	When you have conventions that apply across multiple feature
research.md	When you explored alternatives worth documenting
Full EARS notation	When requirements need to be very precise (complex systems,

---

Next Steps

- PO Guide to SDD — If you're a Product Owner or Product Manager
- Writing Requirements — How to write requirements in different notations
- Brownfield Adoption — If you're adding SDD to an existing codebase
- Greenfield Setup — If you're starting a new project from scratch
- Adoption Guide — The full incremental adoption path

PO Guide to SDD

How Product Owners and Product Managers use Spec-Driven Development. This guide explains your role, your artifacts, and how to write specs that work for both human developers and AI agents.

Your Role in SDD

As PO/PM, you own the *why* and the *what*. You don't need to understand how the code works — but you need to express what it should do with enough precision that two audiences can act on it:

- **Human developers** need context, rationale, and the big picture
- **AI agents** need precision, explicit boundaries, and testable criteria

The good news: writing for AI makes you a better writer for humans too. Vague specs fail with both audiences.

Your Artifacts

Product Brief (brief.md)

Your starting point. Captures the problem, not the solution. Use the Product Brief Template.

When to write one:

- The feature involves multiple people (PO + Dev, or PO + Dev + AI)
- The problem needs framing before jumping to requirements
- You want to align stakeholders on *why* before discussing *what*

When to skip:

- You're working solo and the requirements are clear in your head
- It's a small enhancement with obvious scope

Requirements (requirements.md)

Your main deliverable. Structured requirements with acceptance criteria. Use the requirements section of the Feature Spec Template.

Acceptance Criteria

The most important thing you write. Every requirement needs at least one acceptance criterion that is:

- **Binary** — it either passes or fails, no "partially meets"
- **Observable** — you can see or measure the result
- **Independent** — it doesn't secretly depend on another criterion

---

Writing for Two Audiences

What humans need that AI doesn't

- **Why** this feature matters (context, user empathy, business rationale)
- **Trade-offs** you considered (helps devs make aligned decisions when ambiguity arises)
- **Priority signals** (what matters most if we can't do everything)

What AI needs that humans often skip

- **Explicit boundaries** — what the feature must NOT do (humans infer this; AI doesn't)
- **Edge case handling** — what happens when input is invalid, empty, or unexpected
- **Exact formats** — "email" is vague; "valid email per RFC 5322" is precise
- **No assumed context** — everything the AI needs must be written down; it can't ask a colleague

The sweet spot

Write the *why* and *context* for humans. Write the *what* and *criteria* for AI. Both benefit from both.

> *NOTE: > Objective (for humans): "Users who forget their password currently contact support, generating ~200 tickets/month. Self-service reset will eliminate most of these."*
>
> *Requirement (for AI): "WHEN a user clicks 'Forgot Password' and enters a registered email, THE SYSTEM SHALL send a password reset email containing a single-use token that expires after 30 minutes."*
>
> *Acceptance criterion (for both): "Given a registered user, when they request a password reset, then they receive an email within 60 seconds containing a link with a valid token."*

Your Review Gates

You have three review moments in the SDD Workflow:

1. After Brief → Before Specify

Dev reviews your brief for feasibility. Be open to:

- Scope adjustments (some things may be harder than expected)
- Clarifying questions (your brief will have gaps — that's normal)
- Splitting the feature (one big feature → two focused specs)

2. After Design → Before Task

Dev shares their design. Review it for:

- **Alignment with intent** — Does the design solve the actual problem?
- **Missing requirements** — Did the design reveal something you forgot to specify?
- **Scope drift** — Is the design doing more (or less) than what was required?

You don't need to understand every technical detail. Focus on: "If built as described, will this solve the user's problem?"

3. After Implement → Validate

Walk through each acceptance criterion:

- Does the implementation actually solve the original problem?
- Would a user be satisfied with this?
- Are there edge cases you didn't think of?

If something's off, update the spec and create new tasks — don't just verbally ask for changes.

Common PO Anti-Patterns in SDD

Anti-pattern	Problem	Fix
"Make it intuitive"	Not testable. AI can't evaluate intuitiveness.	Define specific UX behaviors: "Autocomplete suggestio
"Like competitor X"	Requires external research. AI may not know c	Describe the specific behaviors you want, not where yo
"It should be fast"	Not measurable.	"Page loads in under 2 seconds on 3G connection."

Assumed context	"Same as the login flow"	Spell out the specific behaviors — the AI doesn't remem
Scope left open	"And maybe we could also..."	Use the explicit In Scope / Out of Scope sections in the
Over-specifying how	"Use React Query for caching"	Unless it's a hard constraint, leave tech decisions to th

---

Choosing a Notation

See Writing Requirements for full details. Quick guidance:

Notation	Best for	Example
User stories	Capturing user intent and empathy	"As a forgetful user, I want to reset my password via em
EARS notation	Precise system behavior, complex logic	"WHEN the token expires THE SYSTEM SHALL displa
Given/When/Then	Acceptance criteria, test scenarios	"Given expired token, when user clicks link, then show
Plain language	Simple, obvious requirements	"The reset email must include the user's first name"

You can mix notations in a single spec. Use whichever is clearest for each requirement.

Quick Reference

- Start with a Product Brief (problem, need, metrics, scope)
- Expand into requirements with acceptance criteria
- Review the design for alignment with intent
- Validate the implementation against your criteria
- Update the spec if reality changed your understanding

See From Story to Spec for a step-by-step worked example.

From Story to Spec

A step-by-step guide for translating a product idea into a complete SDD spec. Includes a worked example.

The Translation Path

```
Product Idea → Brief → Requirements → Design → Tasks → Implementation
     (PO)       (PO)     (PO + Dev)     (Dev)     (Dev)     (AI + Dev)
```

Each step adds precision. The PO/PM drives the first three steps; the Developer takes over for design and tasks.

Step 1: Start with the Problem

Before writing anything formal, answer these questions:

- **Who** has this problem?
- **What** is the problem? (Describe the pain, not the solution)
- **Why** does it matter now?
- **How** will we know it's solved?

If you can't answer these clearly in conversation, you're not ready to write a spec.

Step 2: Write the Product Brief

Use the Product Brief Template. Capture:

- The problem and who it affects
- What success looks like
- What's in scope and what's out
- What you don't know yet (open questions)

The brief is your thinking tool. It doesn't need to be polished — it needs to be honest about what you know and what you don't.

Step 3: Expand into Requirements

For each thing the system needs to do, write a requirement with an acceptance criterion.

Choose your notation per requirement:

- Use **user stories** when the user's perspective matters
- Use **EARS notation** when system behavior must be precise
- Use **Given/When/Then** for acceptance criteria
- Use **plain language** when the requirement is straightforward

The test: could someone who doesn't know your project read each requirement and understand exactly what to build and how to verify it?

Step 4: Define Boundaries

What must the implementation always do? What must it ask about? What must it never do?

This is where PO/PMs add the most value for AI agents. Developers think about technical boundaries; PO/PMs think about product, data, and experience boundaries.

Step 5: Hand Off to Dev

The Developer takes the requirements and:

- Validates feasibility
- Writes the design document
- Breaks the design into tasks
- Executes with AI assistance

Your job shifts to review and validation.

Worked Example: Password Reset

The Idea

"Users should be able to reset their password themselves instead of contacting support."

Step 1: The Problem

- **Who:** Registered users who forgot their password
- **What:** They can't log in and must email support, waiting 24-48 hours
- **Why now:** ~200 support tickets/month, 35% of total support volume
- **Success:** 80% reduction in password reset tickets

Step 2: The Brief

Problem: Registered users who forget their password must contact support for a manual reset, generating ~200 tickets/month (35% of support volume). Average resolution time is 24-48 hours.

User Need: Users need to reset their password immediately, without waiting for support.

Success Metrics:

- Reduce password reset support tickets by 80%
- 95% of users complete the reset flow without contacting support
- Average time from "forgot password" to "logged in" under 5 minutes

In Scope: Email-based password reset for registered users.

Out of Scope: SMS-based reset, account recovery for unregistered emails, admin password reset.

Open Questions:

- What's our email deliverability rate? (May affect success metrics)
- Do we need to support multiple email addresses per account?

Step 3: The Requirements

Requirement 1 — User Story:

As a registered user who forgot my password, I want to reset it via email so that I can regain access to my account without contacting support.

Acceptance criteria:

- Given a registered email, when the user requests a reset, then they receive an email within 60 seconds
- Given an unregistered email, when the user requests a reset, then the system shows the same confirmation message (no email enumeration)

Requirement 2 — EARS:

WHEN a user submits the password reset form with a registered email, THE SYSTEM SHALL generate a single-use token, store it with a 30-minute expiry, and send an email containing a reset link.

Acceptance criteria:

- Token expires after 30 minutes
- Token is single-use (invalidated after first use)
- Email contains a direct link to the reset page with the token embedded

Requirement 3 — EARS:

WHEN a user submits a new password via a valid reset link, THE SYSTEM SHALL update the password, invalidate the token, invalidate all existing sessions, and redirect to the login page.

Acceptance criteria:

- New password must meet existing password policy
- All existing sessions for the user are terminated
- User can log in with the new password immediately

Requirement 4 — EARS (unwanted behavior):

IF a user submits a new password via an expired or already-used token, THE SYSTEM SHALL display an error message and offer to resend the reset email.

Step 4: The Boundaries

Always:

- Rate-limit reset requests to 3 per email per hour
- Use the same confirmation message for registered and unregistered emails

Never:

- Never reveal whether an email is registered (prevent enumeration)
- Never store the token in plaintext
- Never send the password itself via email

Key Takeaways

- **Start vague, get precise** — The brief is intentionally loose; requirements are intentionally tight
- **One requirement, one behavior** — Don't bundle multiple behaviors into a single requirement
- **Acceptance criteria are the contract** — If it's not in the criteria, it's not required; if it is, it must be verified
- **Boundaries prevent surprises** — Especially important for AI agents that will take your spec literally
- **The PO owns steps 1-4; the Dev owns what comes after** — Clear handoff, clear ownership

Writing Requirements

How to write requirements that work for human developers and AI agents. SDD supports multiple notation formats — choose the one that fits each requirement.

The Four Notations

1. User Stories

Best for: Capturing user intent, building empathy, communicating the "why."

Format:

```
As a [role], I want [goal], so that [benefit].
```

Example:

```
As a mobile user, I want to scan a QR code to join a meeting,
so that I don't have to type a long meeting URL on my phone.
```

When to use:

- The user's perspective is central to the requirement
- You want to communicate why this matters, not just what it does
- The requirement is high-level and will be refined with acceptance criteria

Limitation: User stories alone are often too vague for AI agents. Always pair them with specific acceptance criteria.

2. EARS Notation

Best for: Precise system behavior, complex conditional logic, compliance requirements.

EARS (Easy Approach to Requirements Syntax) uses five patterns:

Pattern	Template	Use when
Ubiquitous	THE SYSTEM SHALL [behavior]	Behavior that always applies

Event-driven	WHEN [event] THE SYSTEM SHALL [response	Behavior triggered by an event
State-driven	WHILE [state] THE SYSTEM SHALL [behavior]	Behavior that applies during a state
Conditional	IF [condition] THEN THE SYSTEM SHALL [beh	Behavior that depends on a condition
Unwanted	IF [unwanted condition] THE SYSTEM SHALL	Handling error or edge cases

Examples:

```
WHEN a user uploads a file larger than 10MB
THE SYSTEM SHALL reject the upload and display a message
stating the maximum file size.

WHILE the system is in maintenance mode
THE SYSTEM SHALL redirect all requests to the maintenance page
and return HTTP 503.

IF the payment gateway returns a timeout error
THE SYSTEM SHALL retry the request once after 5 seconds
and display a failure message if the retry also fails.
```

When to use:

- System behavior must be unambiguous
- There are multiple conditions or states to handle
- The requirement will be implemented by an AI agent with no further clarification
- Regulatory or compliance requirements that must be precise

3. Given/When/Then (Acceptance Criteria)

Best for: Defining testable scenarios, specifying expected behavior for specific cases.

Format:

```
Given [precondition/context]
When [action/event]
Then [expected outcome]
```

Example:

```
Given a user with an expired subscription
When they attempt to access premium content
Then they see an upgrade prompt with their last plan details
And they can still access free content
```

When to use:

- Writing acceptance criteria for any requirement (pairs with all other notations)
- Defining test scenarios
- The requirement is best expressed as a specific scenario

4. Plain Language

Best for: Simple, obvious requirements that don't benefit from formal structure.

Example:

```
The password reset email must include the user's first name in the greeting.
The API must return responses in JSON format.
All timestamps must be in UTC.
```

When to use:

- The requirement is straightforward and unambiguous without structure
- Adding notation would be ceremony without value

Mixing Notations

You can and should mix notations within a single spec. A common pattern:

- **User story** — Sets the context and intent
- **EARS requirements** — Define specific system behaviors
- **Given/When/Then** — Specify acceptance criteria for each requirement
- **Plain language** — Capture simple constraints

> *NOTE: > User Story: As an admin, I want to bulk-invite users via CSV upload, so that I can onboard large teams quickly.*
>
> *Requirement (EARS): WHEN an admin uploads a valid CSV file, THE SYSTEM SHALL create accounts for all listed email addresses and send invitation emails within 5 minutes.*
>
> *Acceptance Criteria (Given/When/Then):*
> *- Given a CSV with 100 valid emails, when uploaded, then all 100 accounts are created and emails sent*
> *- Given a CSV with duplicate emails, when uploaded, then duplicates are skipped and a summary is shown*
> *- Given a CSV with invalid format, when uploaded, then the file is rejected with a specific error message*
>
> *Constraint (Plain): Maximum CSV file size is 1MB. Maximum 500 users per upload.*

The Quality Checklist

For every requirement, verify:

- **Testable** — Can you write a pass/fail check?
- **Unambiguous** — Would two people interpret it the same way?
- **Independent** — Does it stand on its own?
- **Has acceptance criteria** — At least one per requirement
- **No implementation details** — Says *what*, not *how* (unless it's a hard constraint)

For PO/PMs

See PO Guide to SDD for guidance on choosing notations and common anti-patterns. See From Story to Spec for a complete worked example.

Writing Design Docs

How to write the `design.md` for a feature spec. The design document translates requirements into a technical blueprint that an AI agent can implement.

Purpose

The design document answers: **How will we build this?** It sits between requirements (what to build) and tasks (the work to do). Every design decision should trace back to at least one requirement.

Owner: Developer

Reviewers: PO/PM (alignment with intent), other developers (technical soundness)

Structure

A design document covers these sections. Include only what's relevant — not every feature needs every section.

Architecture

How does this feature fit into the existing system? Which components are involved? Draw the high-level picture.

For AI agents, be explicit about:

- Which existing files/modules will be modified
- Which new files/modules will be created
- How new code connects to existing code

Components

What are the building blocks? For each component:

- **Responsibility** — What it does (single responsibility)
- **Interface** — What goes in, what comes out
- **Dependencies** — What it needs from other components
- **Errors** — What can go wrong and how it's handled

Data Model

New or modified data structures. Be specific:

- Field names, types, and constraints
- Relationships between entities
- Migration approach for existing data (if applicable)

API / Interface Contracts

If the feature involves APIs or interfaces:

- Endpoints, methods, request/response shapes
- Authentication/authorization requirements
- Rate limits or quotas
- Error response format

Error Handling

How errors are detected, reported, and recovered from. Cover:

- Expected error cases and their user-facing messages
- Unexpected errors and fallback behavior
- Retry strategies (if applicable)

Testing Strategy

What types of tests are needed:

- Unit tests for business logic
- Integration tests for component interactions
- End-to-end tests for user-facing flows
- What specifically each test type should verify

Writing for AI Agents

Design documents are one of the most important context sources for AI agents during implementation. Tips:

Be explicit about file paths. "Create a new service" is vague. "Create `src/features/password-reset/reset-service.ts`" is actionable.

Name existing utilities to reuse. If there's an existing email sending utility, reference it: "Use the existing `sendEmail()` from `src/lib/email.ts`". This prevents the AI from creating duplicate code.

Describe interfaces precisely. AI agents work best when they know exact function signatures, parameter types, and return types.

State what NOT to do. "Do not create a new database table — extend the existing `users` table" prevents common AI mistakes.

Design Size

Follow the smarter specs, not longer specs principle:

- A typical design document is 1-2 pages
- If it exceeds 3 pages, consider splitting the feature
- Focus on decisions and interfaces, not implementation details (those belong in the code)

Traceability

Every design decision should connect to a requirement:

Requirement	Design Decision
"Token expires after 30 minutes"	Store expires_at timestamp in the password_reset_tokens tabl
"All sessions invalidated on reset"	Call sessionStore.invalidateAll(userId) after password updat

If a design decision doesn't trace to a requirement, either:

- There's a missing requirement (add it)
- The decision is unnecessary (remove it)

Breaking Down Tasks

How to decompose a design into atomic, ordered implementation tasks. Good task decomposition is the difference between smooth AI execution and chaotic back-and-forth.

What Makes a Good Task

Each task should be:

Property	Meaning	Test
Atomic	One logical change	Could you describe it in one sentence?
Ordered	Dependencies flow forward	Does it only depend on earlier tasks?
Testable	Independently verifiable	Can you check it works without completing later tasks?
Specific	No interpretation needed	Would two developers implement it the same way?
Scoped	Clear file boundaries	Do you know which files are touched?

Decomposition Strategy

Start from the Design

Walk through the design document and identify:

- Data model changes (these come first — everything depends on them)
- Core business logic (the main feature behavior)
- Integration points (connecting to existing systems)
- User-facing components (UI, API endpoints)
- Error handling and edge cases
- Tests (can be interleaved or grouped at the end)

Order by Dependency

A common ordering pattern:

- Infrastructure / data model setup
- Core logic / service layer
- API routes or handlers
- UI components
- Integration and wiring
- Error handling refinement
- Tests and validation

Size Check

A well-sized task is roughly **one commit**. If describing the task takes more than 3-4 lines, it's probably too large — split it.

Task Format

Use the Task List Template format:

```
### 1. Create password reset token table

- [ ] **Status:** Not started
- **Description:** Add a `password_reset_tokens` table with columns:
  id (UUID), user_id (FK), token_hash (varchar), expires_at (timestamp),
  used_at (nullable timestamp). Add index on token_hash.
- **Files:** `src/db/migrations/xxx-create-password-reset-tokens.ts`
- **Depends on:** None
- **Validates:** Requirement 2 (token storage with expiry)
```

Common Mistakes

Tasks too large:

- "Implement the password reset feature" — This is the entire spec, not a task
- Split into: create table, create service, create API endpoint, create email template, create UI form, etc.

Tasks too small:

- "Add import statement for uuid library" — This isn't a logical unit of work

- Group related micro-changes into one meaningful task

Missing dependencies:

- Task 5 modifies a file created in task 3, but doesn't list the dependency
- Always check: does this task assume something from an earlier task?

Vague descriptions:

- "Update the API" — Which endpoint? What change? What's the expected behavior?
- Be specific: "Add POST `/api/password-reset/request` endpoint that accepts `{ email: string }` and returns 200 with `{ message: 'If the email is registered...' }`"

Working with AI Agents

When an AI agent helps decompose tasks:

- Provide the full design document as context
- Review the proposed task list before execution
- Check that task order matches dependency flow
- Verify that all requirements are covered (each requirement should map to at least one task)

When an AI agent executes tasks:

- The agent reads the full spec context (steering + spec folder)
- It executes tasks in order, one at a time
- After each task, the developer reviews before proceeding
- If a task reveals a design issue, update the spec — don't hack around it

---

How Many Tasks?

Task Count	Signal
1-4	Very small feature — consider if you even need a spec
5-15	Sweet spot for most features
15+	Feature is probably too large — consider splitting into mult

Spec Review Checklist

Use this checklist to validate a spec before moving to implementation. Review at each phase gate in the Workflow.

After Brief (Phase 0 → Phase 1)

- Problem is clearly stated from the user's perspective
- Success metrics are defined and measurable
- Scope boundaries are explicit (in scope and out of scope)
- Open questions are listed (not hidden assumptions)
- Developer has reviewed for feasibility

After Requirements (Phase 1 → Phase 2)

- Every requirement has at least one acceptance criterion
- Acceptance criteria are testable (pass/fail, not subjective)
- No implicit assumptions — everything needed is written down
- Requirements describe *what*, not *how*
- Boundaries are defined (Always/Ask/Never)
- No duplicate or contradictory requirements
- A developer unfamiliar with the project could understand what to build
- An AI agent could implement without asking clarifying questions

After Design (Phase 2 → Phase 3)

- Every design decision traces to at least one requirement
- No design decision exists without a corresponding requirement
- File paths and component names are specific
- Existing utilities and patterns are referenced (no unnecessary new code)
- Error handling is covered
- Testing strategy is defined
- PO/PM confirms design aligns with product intent

After Tasks (Phase 3 → Phase 4)

- Tasks are atomic (one logical change each)
- Tasks are ordered by dependency
- Every task lists which files it touches
- Every task references which requirement it satisfies
- All requirements are covered by at least one task
- Task count is between 5-15 (if not, reconsider scope)
- A developer could execute each task without ambiguity

After Implementation (Phase 4 → Phase 5)

- Every acceptance criterion has been verified
- No requirements were skipped or only partially implemented
- Boundaries were respected (check Always/Ask/Never compliance)
- Spec was updated if implementation revealed new information
- PO/PM validates the feature solves the original problem
- Developer validates technical quality and absence of regressions

Brownfield Adoption

How to adopt SDD in an existing codebase — incrementally, without disrupting ongoing work.

Start Where You Are

You don't need to retroactively spec everything. SDD is adopted **going forward**, one feature at a time.

Week 1: Steering Document

Create your steering document at the project root. Focus on:

- Project overview (what this is)
- Tech stack (what you use)
- Key commands (build, test, lint)
- 3-5 boundaries (the most important Always/Ask/Never rules)

This alone improves AI-assisted development — the agent now has context about your project.

Week 2+: First Spec

Pick a feature that's:

- **Medium complexity** — complex enough to benefit from planning, simple enough that spec-writing doesn't feel overwhelming
- **Upcoming** — something you're about to build, not something already built
- **Self-contained** — minimal dependencies on other in-progress work

Create `specs/[feature-name]/` and write requirements, design, and tasks. See Getting Started for the step-by-step.

Ongoing: Build the Habit

After 2-3 features with specs:

- You'll know which parts of SDD help your team most
- You'll develop your own conventions (add them to the steering document)
- You'll recognize when a feature needs a spec vs. when it doesn't

---

What About Existing Code?

Don't retroactively spec existing features. That's busywork with no payoff.

Do spec when modifying existing features:

- If you're making significant changes to an existing feature, write a spec for the changes
- Reference the existing code in your design document
- The spec covers the *delta*, not the entire feature

Do document discovered conventions as standards:

- If you notice patterns in the existing code, capture them in `standards/[topic].md`
- This helps AI agents follow existing conventions when making changes

---

Common Challenges

Challenge	Solution
"We don't have a PO"	The person who decides what to build plays the PO role. It m
"Our codebase is messy"	SDD doesn't require clean code. Specs help you make focused,
"We use agile sprints"	SDD fits inside sprints. A spec can be written and executed
"Our team is skeptical"	Start with one person, one feature. Let the results speak. D
"We already have docs"	Great. Your existing docs may become your steering document

---

Integration with Existing Processes

SDD overlaps with — but doesn't replace — these common practices:

Practice	Relationship to SDD
Agile user stories	User stories become one notation option in requirements.md
Technical design docs	Design docs become design.md in the spec folder
PR descriptions	Specs provide richer context than PR descriptions; link spec
ADRs (Architecture Decision Records)	ADRs cover project-wide decisions; design.md covers feature-
JIRA/Linear tickets	Tickets can reference spec folders; tasks.md provides more d

Greenfield Setup

How to start a new project with SDD from day one.

Initial Setup

1. Create the Steering Document

Before writing any code, create your steering document. Use the Steering Template.

For a new project, focus on:

- **Project overview** — What you're building and why
- **Tech stack** — Lock in your choices early; be specific about versions
- **Project structure** — Define where files go before any exist
- **Boundaries** — Start with sensible defaults (see below)

2. Create the Specs Folder

```
mkdir specs standards
```

3. Define Initial Standards (optional)

If you have strong opinions about code style, error handling, or API conventions, capture them in `standards/` now. These will guide AI agents from the first line of code.

Recommended Starting Boundaries

For new projects, these boundaries work well as defaults:

Always:

- Follow the project structure defined in the steering document
- Write tests for new functionality

- Use existing utilities before creating new ones
- Keep changes focused on the current task

Ask:

- Before adding any new dependency
- Before creating a new top-level directory
- Before changing the project structure

Never:

- Commit secrets, credentials, or API keys
- Skip or disable tests
- Create files outside the defined project structure

Adjust these as your project evolves.

First Feature

For your very first feature, the full SDD workflow might feel like overhead. A pragmatic approach:

- **Start with the steering document** — this provides project-wide context
- **Write a brief spec** for your first feature — even a lightweight one helps
- **Use the spec to bootstrap the project** — the first spec defines the project's initial structure, data model, and core patterns
- **Increase spec rigor as the project grows** — early specs can be lighter; later specs should be more thorough as there's more existing code to protect

---

Project Structure Convention

SDD recommends this top-level structure (adapt to your framework):

```
project-root/
  CLAUDE.md                   # Steering document
  specs/                      # SDD spec folders
    [feature-name]/
      brief.md
      requirements.md
      design.md
      tasks.md
  standards/                  # Project-wide coding standards
    [topic].md
  src/                        # Source code (or your framework's convention)
  tests/                      # Test files (if separate from src)
```

The key principle: `specs/` and `standards/` are top-level, version-controlled, and visible. They are first-class project artifacts, not hidden metadata.

From Zero to First Commit

A practical sequence for a new project:

- Initialize the repository
- Create the steering document (`CLAUDE.md`)
- Create `specs/` and `standards/` directories
- Write a brief for your first feature
- Write requirements and design
- Write tasks
- Execute tasks (this produces your first code)
- Validate and update the spec

Your first commit should include the steering document and your first spec alongside the initial code. This establishes the pattern for the entire project.

Part: Templates

Copyable templates for your projects

Product Brief Template

> *NOTE: Copy this template into your project at specs/[feature-name]/brief.md. This is the PO/PM's starting point — capture intent before formal requirements.*

[Feature Name] — Product Brief

Problem Statement

[What user pain, business need, or opportunity are we addressing? Be specific about who is affected and what the current situation is.]

> *NOTE: Write from the user's perspective. "Users currently have to contact support to reset their password, which takes 24-48 hours and generates ~200 tickets/month."*

User Need

[Who is the primary user? What do they need to be able to do? What is their current workaround, if any?]

Success Metrics

[How will we measure whether this feature succeeded? Include at least one quantitative metric.]

- [Metric 1: e.g., "Reduce password reset support tickets by 80%"]
- [Metric 2: e.g., "95% of users complete the reset flow without contacting support"]

> *NOTE: If you can't define a success metric, the problem may not be well-understood enough yet. That's okay — list it as an open question below.*

Scope

In Scope

- [What this feature will do]
- [What this feature will do]

Out of Scope

- [What this feature explicitly will NOT do]
- [What this feature explicitly will NOT do]

> *NOTE: Being explicit about what's out of scope is as important as defining what's in. It prevents scope creep and gives the AI agent clear boundaries.*

Constraints

[Known limitations, deadlines, dependencies, regulatory requirements, or technical constraints that the team must work within.]

- [Constraint 1]
- [Constraint 2]

Open Questions

[What the PO/PM doesn't know yet and needs answered before or during the Specify phase.]

- [Question 1]
- [Question 2]

> *NOTE: Next step: Once this brief is reviewed by the Dev team, expand it into a formal requirements.md using the Feature Spec Template. Delete all > NOTE: callouts when you're done filling in the template.*

Steering Template

> *NOTE: Copy this template to your project root as CLAUDE.md, .cursorrules, .github/copilot-instructions.md, or whatever your AI tool uses. This is your project's constitution — it applies to all work in the project.*

[Project Name]

Project Overview

[One paragraph describing what this project is, its purpose, and its current state.]

Tech Stack

[Be specific — include versions where they matter.]

- **Language:** [e.g., TypeScript 5.4]
- **Framework:** [e.g., Next.js 14 with App Router]
- **Database:** [e.g., PostgreSQL 16 via Prisma ORM]
- **Testing:** [e.g., Vitest for unit tests, Playwright for e2e]
- **Other:** [e.g., Tailwind CSS, Redis for caching]

Commands

```
# Build
[build command]

# Test
[test command]

# Lint
[lint command]

# Dev server
[dev command]
```

Project Structure

```
[Key directories and their purpose — keep this high-level]
src/
  components/     # Shared UI components
  features/       # Feature-specific code, organized by feature
  lib/            # Shared utilities and helpers
  api/            # API routes
specs/            # SDD spec folders
standards/        # Project-wide coding standards
```

Code Conventions

[Naming, formatting, patterns, and architectural rules.]

- [Convention 1: e.g., "Use camelCase for variables, PascalCase for components"]
- [Convention 2: e.g., "All API responses follow the { data, error, meta } shape"]
- [Convention 3: e.g., "Feature code lives in src/features/[feature-name]/"]

Boundaries

Always

- [e.g., "Run tests before completing any task"]
- [e.g., "Follow the existing file and folder structure"]
- [e.g., "Use the existing database schema"]

Ask

- [e.g., "Before adding a new dependency"]
- [e.g., "Before changing a public API contract"]
- [e.g., "Before modifying shared configuration"]

Never

- [e.g., "Never modify .env files or commit secrets"]
- [e.g., "Never push directly to main"]
- [e.g., "Never skip or disable tests"]
- [e.g., "Never modify CI/CD pipeline configuration"]

> *NOTE: Delete all > NOTE: callouts and placeholder text when you're done. The steering document should contain only your project's actual conventions.*

Feature Spec Template

> *NOTE: This template covers all three core spec files for a feature. Create a folder at specs/[feature-name]/ and split this into separate files: requirements.md, design.md, and tasks.md. Optionally start with a brief.md using the Product Brief Template.*

---

requirements.md

[Feature Name] — Requirements

Objective

[One paragraph: what problem does this solve and why now? Reference the Product Brief if one exists.]

Requirements

> *NOTE: Choose the notation that fits each requirement. You can mix notations within a single spec. See Writing Requirements for guidance on when to use each format.*

User Stories

As a [role], I want [goal], so that [benefit].

Acceptance criteria:

- Given [context], when [action], then [outcome]
- Given [context], when [action], then [outcome]

System Behavior

WHEN [trigger event]

THE SYSTEM SHALL [expected response]

Acceptance criteria:

- [Testable criterion]

Conditional Requirements

IF [condition]

THEN THE SYSTEM SHALL [behavior]

Acceptance criteria:

- [Testable criterion]

Boundaries

Always

- [Feature-specific things the implementation must always do]

Ask

- [Feature-specific things requiring human confirmation]

Never

- [Feature-specific prohibitions]

Acceptance Criteria Summary

- [Criterion 1]
- [Criterion 2]
- [Criterion 3]

---

design.md

[Feature Name] — Design

Architecture

[How does this feature fit into the existing system? Which components are involved?]

Components

[What new or modified components are needed? Define their responsibilities and interfaces.]

[Component Name]

Responsibility: [What it does]

Input: [What it receives]

Output: [What it produces]

Errors: [What can go wrong]

Data Model

[New or modified data structures, database tables, API contracts.]

Integration Points

[How this feature connects to existing systems, external APIs, or other features.]

Error Handling

[How errors are detected, reported, and recovered from.]

Testing Strategy

[What types of tests are needed and what they verify.]

tasks.md

[Feature Name] — Tasks

Task List

> *NOTE: Tasks should be atomic (one logical change), ordered by dependency, and independently testable. Mark status as: [ ] not started, [-] in progress, [x] complete.*

1. [Task title]

- **Status:** Not started
- **Description:** [What to do, specifically]
- **Files:** [Which files to create or modify]
- **Validates:** [Which requirement/acceptance criterion this satisfies]

2. [Task title]

- **Status:** Not started
- **Description:** [What to do, specifically]
- **Files:** [Which files to create or modify]
- **Depends on:** Task 1
- **Validates:** [Which requirement/acceptance criterion this satisfies]

3. [Task title]

- **Status:** Not started
- **Description:** [What to do, specifically]
- **Files:** [Which files to create or modify]
- **Depends on:** Task 2
- **Validates:** [Which requirement/acceptance criterion this satisfies]

> *NOTE: Delete all > NOTE: callouts and replace all [BRACKETS] with your actual content. See Anatomy of a Spec for quality guidelines.*

Bugfix Spec Template

> *NOTE: Create a folder at specs/[bugfix-name]/ and split this into two files: bugfix.md and tasks.md. Use this for bugs where regressions are costly or the fix requires investigation.*

---

bugfix.md

[Bug Name] — Bugfix Analysis

Bug Summary

[One paragraph: what's broken, who is affected, and how severe is it?]

Current Behavior (Defective)

[What happens now. Be precise — include steps to reproduce.]

- [Step 1]
- [Step 2]
- [Observed result — what goes wrong]

Expected Behavior (Correct)

[What should happen instead. Be specific enough that this becomes a test case.]

- [Step 1]
- [Step 2]
- [Expected result]

Unchanged Behavior (Regression Prevention)

[What existing functionality must NOT be affected by the fix. These become regression test cases.]

- [Behavior 1 that must continue working]
- [Behavior 2 that must continue working]

> *NOTE: This section is critical. It prevents the fix from breaking something else. Think about what users currently do that works fine and must keep working.*

Root Cause Analysis

[What is causing the defect? If unknown, describe your investigation approach.]

> *NOTE: This section may be filled in during investigation. It's okay to start with a hypothesis and update it.*

Proposed Fix

[High-level description of the fix approach. Not the code — the strategy.]

Boundaries

Always

- [e.g., "Preserve existing API contract"]

Never

- [e.g., "Never modify the user table schema"]

---

tasks.md

[Bug Name] — Fix Tasks

1. Reproduce the bug

- **Status:** Not started
- **Description:** Create a reliable reproduction of the defective behavior
- **Validates:** Current Behavior section

2. Implement the fix

- **Status:** Not started
- **Description:** [Specific fix based on root cause analysis]
- **Files:** [Which files to modify]
- **Validates:** Expected Behavior section

3. Verify regression prevention

- **Status:** Not started
- **Description:** Confirm all unchanged behaviors still work
- **Validates:** Unchanged Behavior section

4. [Additional tasks as needed]

- **Status:** Not started
- **Description:** [What to do]
- **Depends on:** [Previous task]

> *NOTE: Delete all > NOTE: callouts when done. For simple bugs, tasks 1-3 may be all you need.*

Standards Template

> *NOTE: Copy this template to standards/[topic].md in your project. Use it to define a project-wide coding convention or architectural standard. File names should be hyphenated slugs (e.g., error-handling.md, api-conventions.md).*

[Standard Name]

Purpose

[Why does this standard exist? What problem does it prevent?]

The Standard

[Clear, prescriptive rules. Write these as instructions the AI agent and developers will follow.]

[Rule 1 Name]

[Description of the rule with rationale.]

```
[Code example showing the correct approach]
```

[Rule 2 Name]

[Description of the rule with rationale.]

```
[Code example showing the correct approach]
```

Anti-patterns

[What NOT to do, with examples.]

```
[Code example showing the incorrect approach]
```

Why this is wrong: [Brief explanation]

Exceptions

[When is it acceptable to deviate from this standard? What approval is needed?]

> *NOTE: Keep standards concise. Each standard file should cover one topic and fit on one screen. If it's getting long, consider splitting it.*

Task List Template

> *NOTE: Copy this template to specs/[feature-name]/tasks.md in your project. Tasks should be atomic, ordered by dependency, and independently testable.*

[Feature/Bugfix Name] — Tasks

Spec: requirements | design

Status: Not started | In progress | Complete

Task List

1. [Task title]

- **Status:** Not started
- **Description:** [What to do — specific enough that no interpretation is needed]
- **Files:** [Which files to create or modify]
- **Depends on:** None
- **Validates:** [Which requirement or acceptance criterion this satisfies]

2. [Task title]

- **Status:** Not started
- **Description:** [What to do]
- **Files:** [Which files to create or modify]
- **Depends on:** Task 1
- **Validates:** [Which requirement or acceptance criterion this satisfies]

3. [Task title]

- **Status:** Not started
- **Description:** [What to do]
- **Files:** [Which files to create or modify]

- **Depends on:** Task 2
- **Validates:** [Which requirement or acceptance criterion this satisfies]

Progress

- Total tasks: [N]
- Completed: [0]
- Remaining: [N]

> *NOTE: Status markers: [ ] not started, [-] in progress, [x] complete. Update these as tasks are executed. A good task list has 5-15 tasks. More than 15 suggests the feature should be split into smaller specs.*

Part: Examples

Fully worked examples demonstrating SDD in practice

Examples

Fully worked examples demonstrating the SDD standard in practice. Each example mirrors the exact file structure that a real project would use.

Feature Spec Example: Notification Preferences

A complete feature spec for a fictional **TaskFlow** application (see TaskFlow steering document). Demonstrates the full 6-phase workflow from Product Brief through implementation tasks.

Folder structure:

```
Examples/notification-preferences/
  brief.md            ← PO/PM starting artifact
  requirements.md     ← Structured requirements + acceptance criteria
  design.md           ← Architecture and technical design
  tasks.md            ← Ordered atomic implementation tasks
```

Files:

- brief.md — Product Brief capturing user problem, success metrics, scope
- requirements.md — 5 requirements using mixed notation (user stories + EARS), with acceptance criteria and boundaries
- design.md — Component architecture, data model, API contracts, error handling
- tasks.md — 7 ordered tasks with dependencies and requirement traceability

---

Bugfix Spec Example: Duplicate Order Emails

A bugfix spec for a duplicate email sending issue. Demonstrates the bugfix variant with current/expected/unchanged behavior analysis.

Folder structure:

```
Examples/duplicate-order-emails/
  bugfix.md           ← Bug analysis (replaces requirements.md)
  tasks.md            ← Fix implementation tasks
```

Files:

- bugfix.md — Bug summary, reproduction steps, root cause analysis, proposed fix, boundaries
- tasks.md — 4 tasks: reproduce, fix backend, fix UI, verify regressions

---

Steering Document Example: TaskFlow

A complete project steering document for a fictional SaaS task management application.

Folder structure:

```
Examples/taskflow-steering/
  CLAUDE.md            ← The steering document itself
```

File:

- CLAUDE.md — Project overview, tech stack, commands, project structure, code conventions, Always/Ask/Never boundaries

---

How These Examples Map to Your Project

In a real project, the structure would look like:

```
your-project/
  CLAUDE.md                           ← Like taskflow-steering/CLAUDE.md
  specs/
    notification-preferences/         ← Like Examples/notification-preferences/
      brief.md
      requirements.md
      design.md
      tasks.md
    duplicate-order-emails/           ← Like Examples/duplicate-order-emails/
      bugfix.md
      tasks.md
```

The examples use the same filenames, structure, and conventions prescribed by the standard. Copy a folder, rename it, and replace the content.

Notification Preferences — Product Brief

Problem Statement

Users receive all notification types (email, in-app, push) by default and have no way to control which notifications they receive. This generates complaints (~50/month) and contributes to email unsubscribes (12% monthly unsubscribe rate).

User Need

Users need to choose which types of notifications they receive and through which channels. Power users want granular control; casual users want a simple on/off.

Success Metrics

- Reduce notification-related complaints by 70%
- Reduce email unsubscribe rate from 12% to under 5%
- 60% of active users customize their preferences within 3 months

Scope

In Scope

- Notification preferences UI in account settings
- Per-category and per-channel toggles
- A "mute all" quick action
- Persisting preferences and applying them to notification delivery

Out of Scope

- Notification history / log viewing
- Notification scheduling (e.g., "send only between 9am-5pm")
- SMS as a notification channel (future consideration)

Constraints

- Must work on mobile and desktop
- Must not require re-authentication to change preferences
- Changes must take effect immediately (no batch processing delay)

Open Questions

- How many notification categories do we currently have? (Need inventory)
- Are there any notifications that must always be sent regardless of preferences (e.g., security alerts)?

Notification Preferences — Requirements

Objective

Allow users to control which notifications they receive and through which channels, reducing complaints and unsubscribes while preserving delivery of critical communications.

Requirements

R1: Preference Access

As a logged-in user, I want to access my notification preferences from account settings, so that I can control what notifications I receive.

Acceptance criteria:

- Given a logged-in user, when they navigate to account settings, then they see a "Notification Preferences" section
- Given a logged-in user on mobile, when they access notification preferences, then the UI is fully functional on screen widths from 320px

R2: Per-Category Toggle

WHEN a user views notification preferences

THE SYSTEM SHALL display all notification categories with independent on/off toggles for each channel (email, in-app, push).

Acceptance criteria:

- Each category shows toggles for: email, in-app, push
- Toggles reflect the user's current saved preferences
- Default state for new users: all channels enabled for all categories

R3: Mute All

As a user overwhelmed by notifications, I want a single "Mute All" toggle so that I can quickly silence everything non-critical.

Acceptance criteria:

- Given a user who enables "Mute All", then all non-mandatory notification toggles are set to off

- Given a user who disables "Mute All", then preferences return to their state before muting
- Mandatory notifications (security alerts) remain enabled regardless of "Mute All"

R4: Immediate Effect

WHEN a user changes a notification preference

THE SYSTEM SHALL apply the change immediately to all subsequent notifications without requiring page refresh or re-authentication.

Acceptance criteria:

- Changes are persisted within 1 second
- Next notification respects the updated preference
- No page refresh or re-login required

R5: Mandatory Notifications

IF a notification is categorized as mandatory (security alerts, account actions)

THEN THE SYSTEM SHALL always deliver it regardless of user preferences, and the corresponding toggles SHALL be disabled with an explanatory tooltip.

Acceptance criteria:

- Mandatory category toggles are visually disabled
- Hovering/tapping a disabled toggle shows "This notification cannot be disabled for your security"
- Mandatory notifications are always delivered even when "Mute All" is active

Boundaries

Always

- Preserve the user's pre-mute preferences when "Mute All" is toggled off
- Show mandatory notifications as disabled with explanation (never hide them)

Ask

- Before adding a new notification category to the system

Never

- Never allow mandatory notifications to be disabled
- Never require re-authentication to change preferences

Notification Preferences — Design

Architecture

This feature adds a preferences layer between the notification generation system and the delivery system. The existing `NotificationService` will check preferences before dispatching.

```
User → Preferences UI → Preferences API → preferences table
                                                ↓
Notification triggered → NotificationService → check preferences → deliver (or skip)
```

Components

PreferencesAPI

Responsibility: CRUD operations for user notification preferences

Endpoints:

- `GET /api/users/:id/notification-preferences` → returns all preferences
- `PUT /api/users/:id/notification-preferences` → updates preferences (partial update)
- `POST /api/users/:id/notification-preferences/mute-all` → toggles mute

Errors: 401 (not authenticated), 403 (not own preferences), 422 (invalid category/channel)

PreferencesStore

Responsibility: Database access for notification preferences

Location: `src/features/notifications/preferences-store.ts`

Uses: Existing `db` utility from `src/lib/database.ts`

NotificationPreferencesUI

Responsibility: Settings page component for managing preferences

Location: `src/features/notifications/components/NotificationPreferences.tsx`

Uses: Existing `ToggleSwitch` component from `src/components/ui/`

Data Model

New table: `notification_preferences`

Column	Type	Notes
id	UUID	Primary key
user_id	UUID	FK to users, unique per user+category+channel
category	VARCHAR(50)	e.g., "marketing", "product_updates", "security"
channel	VARCHAR(20)	"email", "in_app", "push"
enabled	BOOLEAN	Default: true
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

Index on `(user_id, category, channel)` — unique constraint.

New column on `notification_categories` table: `mandatory BOOLEAN DEFAULT false`

Integration Points

- `NotificationService.send()` — Add preference check before dispatch. Use existing service at `src/services/notification-service.ts`.
- `src/lib/email.ts` — No changes; preferences are checked before email is triggered.

Error Handling

- If preferences table is unreachable, **deliver the notification** (fail-open for user experience).
- If an invalid category is submitted, return 422 with the list of valid categories.

Testing Strategy

- Unit tests: PreferencesStore CRUD, mute-all logic, mandatory override
- Integration tests: API endpoints with auth
- E2E test: Toggle a preference, trigger a notification, verify delivery/non-delivery

Notification Preferences — Tasks

Spec: requirements | design

Status: Not started

1. Create notification preferences database migration

- **Status:** Not started
- **Description:** Create `notification_preferences` table with columns defined in design. Add `mandatory` boolean column to existing `notification_categories` table. Set `mandatory = true` for "security" and "account_actions" categories.
- **Files:** `src/db/migrations/xxx-notification-preferences.ts`
- **Depends on:** None
- **Validates:** R2 (data model for per-category toggles)

2. Implement PreferencesStore

- **Status:** Not started
- **Description:** Create data access layer with methods: `getByUserId(userId)`, `upsert(userId, category, channel, enabled)`, `muteAll(userId)`, `unmuteAll(userId)`. Use existing `db` utility from `src/lib/database.ts`.
- **Files:** `src/features/notifications/preferences-store.ts`
- **Depends on:** Task 1
- **Validates:** R2, R3, R4

3. Implement PreferencesAPI endpoints

- **Status:** Not started
- **Description:** Create GET, PUT, and POST (mute-all) endpoints as defined in design. Include auth middleware. Return 401/403/422 for error cases. PUT supports partial updates.
- **Files:** `src/features/notifications/preferences-api.ts`, `src/routes/api.ts` (register routes)
- **Depends on:** Task 2
- **Validates:** R1, R2, R3, R4

4. Add preference check to NotificationService

- **Status:** Not started
- **Description:** Before dispatching a notification, check user preferences. Skip delivery if the category+channel is disabled. Always deliver if `mandatory = true` on the category. Fail-open if preferences table is unreachable.
- **Files:** `src/services/notification-service.ts`
- **Depends on:** Task 2

- **Validates:** R4, R5

5. Build NotificationPreferencesUI component

- **Status:** Not started
- **Description:** Create settings page component showing all categories with per-channel toggles. Disable toggles for mandatory categories with tooltip. Include "Mute All" toggle at top. Use existing `ToggleSwitch` from `src/components/ui/`. Responsive from 320px.
- **Files:** `src/features/notifications/components/NotificationPreferences.tsx`
- **Depends on:** Task 3
- **Validates:** R1, R2, R3, R5

6. Implement Mute All with preference preservation

- **Status:** Not started
- **Description:** When "Mute All" is activated, store current preferences snapshot, then set all non-mandatory to disabled. When deactivated, restore from snapshot. Ensure mandatory categories are never affected.
- **Files:** `src/features/notifications/preferences-store.ts`, `src/features/notifications/components/NotificationPreferences.tsx`
- **Depends on:** Task 5
- **Validates:** R3, R5

7. Write tests

- **Status:** Not started
- **Description:** Unit tests for PreferencesStore (CRUD, mute/unmute, mandatory override). Integration tests for API endpoints (auth, validation, partial update). E2E test for toggle → notification delivery flow.
- **Files:** `tests/features/notifications/preferences-store.test.ts`, `tests/features/notifications/preferences-api.test.ts`, `tests/e2e/notification-preferences.test.ts`
- **Depends on:** Task 6
- **Validates:** All requirements

Progress

- Total tasks: 7
- Completed: 0
- Remaining: 7

Duplicate Emails on Order Confirmation — Bugfix Analysis

Bug Summary

Some customers receive duplicate order confirmation emails after completing a purchase. The issue affects approximately 8% of orders and has generated 45 support complaints in the last 2 weeks. Duplicates are exact copies sent 1-3 seconds apart.

Current Behavior (Defective)

- User completes checkout and clicks "Place Order"
- System processes the payment successfully
- System sends an order confirmation email
- **System sends a second identical confirmation email 1-3 seconds later**
- User receives two identical emails

Reproduction: Occurs intermittently. More frequent during high-traffic periods. Reliable reproduction by clicking "Place Order" rapidly twice.

Expected Behavior (Correct)

- User completes checkout and clicks "Place Order"
- System processes the payment successfully
- System sends exactly one order confirmation email
- Subsequent clicks on "Place Order" for the same order are ignored

Unchanged Behavior (Regression Prevention)

- Order confirmation email content and formatting must remain identical
- Email delivery timing must remain within current SLA (under 60 seconds)
- Other transactional emails (shipping, refund) must not be affected
- The checkout flow and payment processing must work identically
- Orders placed through the API (not the web UI) must continue to work

Root Cause Analysis

The order completion handler is called by both the payment webhook and the UI success callback. During normal flow, the payment webhook fires first and sends the email. When the UI callback arrives shortly after, it triggers the email again because there is no idempotency check on email dispatch.

The race condition is amplified by:

- No debounce on the "Place Order" button
- No idempotency key on the order completion handler
- No deduplication check in the email queue

Proposed Fix

Two-layer fix:

- **UI layer:** Disable the "Place Order" button after first click to prevent duplicate submissions
- **Backend layer:** Add an idempotency check to the order completion handler using the order ID. If the confirmation email has already been queued/sent for this order, skip the duplicate.

The backend fix is the primary fix (handles all entry points). The UI fix is defense-in-depth.

Boundaries

Always

- Preserve all existing email content and formatting
- Maintain the current email delivery SLA

Never

- Never modify the payment processing flow
- Never change the webhook handler's response format (external integrations depend on it)

Duplicate Emails on Order Confirmation — Fix Tasks

Spec: bugfix

Status: Not started

1. Reproduce the bug

- **Status:** Not started
- **Description:** Create a test that demonstrates the duplicate email behavior. Simulate both the webhook and UI callback firing for the same order and verify two emails are sent. This test should fail before the fix and pass after.
- **Files:** `tests/orders/duplicate-email.test.ts`
- **Depends on:** None
- **Validates:** Current Behavior section (confirms the bug exists)

2. Add idempotency check to order completion handler

- **Status:** Not started
- **Description:** Add an `email_sent_at` nullable timestamp column to the `orders` table. In the order completion handler, check this column before sending the confirmation email. If already set, skip email dispatch. Use a database transaction to prevent race conditions between concurrent handlers.
- **Files:** `src/db/migrations/xxx-add-email-sent-at.ts`, `src/services/order-service.ts`
- **Depends on:** Task 1
- **Validates:** Expected Behavior (exactly one email per order)

3. Disable Place Order button after first click

- **Status:** Not started
- **Description:** Add loading/disabled state to the "Place Order" button. After first click: disable the button, show a loading indicator. Re-enable only if the request fails. This is defense-in-depth — the backend fix is the primary safeguard.
- **Files:** `src/features/checkout/components/PlaceOrderButton.tsx`
- **Depends on:** None (independent of backend fix)
- **Validates:** Expected Behavior (UI prevents rapid double-click)

4. Verify regression prevention

- **Status:** Not started
- **Description:** Run existing order and email test suites. Add specific tests verifying: (1) single confirmation email is still sent, (2) email content is unchanged, (3) API-placed orders still receive confirmation, (4) shipping and refund emails are unaffected.

- **Files:** `tests/orders/duplicate-email.test.ts` (extend), `tests/orders/order-flow.test.ts` (verify passes)
- **Depends on:** Task 2, Task 3
- **Validates:** Unchanged Behavior section (no regressions)

Progress

- Total tasks: 4
- Completed: 0
- Remaining: 4

TaskFlow

Project Overview

TaskFlow is a SaaS task management application for small teams (2-20 people). It provides boards, lists, and cards for organizing work, with real-time collaboration, due dates, and basic reporting. Currently in active development with 3 developers and ~2,000 active users.

Tech Stack

- **Language:** TypeScript 5.4 (strict mode)
- **Frontend:** Next.js 14 with App Router, React 18
- **Styling:** Tailwind CSS 3.4
- **Backend:** Next.js API routes + tRPC for type-safe API calls
- **Database:** PostgreSQL 16 via Prisma ORM (schema at `prisma/schema.prisma`)
- **Auth:** NextAuth.js v5 with email + Google OAuth
- **Real-time:** Pusher for WebSocket connections
- **Testing:** Vitest for unit/integration, Playwright for e2e
- **Hosting:** Vercel (frontend), Railway (database)

Commands

```
# Install dependencies
pnpm install

# Development server
pnpm dev

# Run all tests
pnpm test

# Run e2e tests
pnpm test:e2e

# Lint and type check
pnpm lint && pnpm typecheck

# Database migrations
pnpm prisma migrate dev

# Generate Prisma client
pnpm prisma generate
```

Project Structure

```
src/
  app/                    # Next.js App Router pages
    (auth)/               # Auth pages (login, register)
    (dashboard)/          # Authenticated pages
    api/                  # API routes
  components/
    ui/                   # Shared UI components (Button, Input, Modal, etc.)
    layout/               # Layout components (Sidebar, Header)
  features/
    boards/               # Board management
    cards/                # Card CRUD and detail views
    lists/                # List management within boards
    users/                # User profiles and team management
    notifications/        # Notification system
  lib/
    database.ts           # Prisma client singleton
    auth.ts               # NextAuth configuration
    pusher.ts             # Pusher client/server setup
    email.ts              # Email sending utility (Resend)
  server/
    trpc/                 # tRPC router definitions
specs/                    # SDD spec folders
standards/                # Project-wide coding standards
prisma/
  schema.prisma           # Database schema
  migrations/             # Migration files
tests/
  e2e/                    # Playwright e2e tests
```

Code Conventions

- **Naming:** camelCase for variables/functions, PascalCase for components/types, SCREAMING_SNAKE for constants
- **Files:** Feature code lives in `src/features/[feature-name]/`. Shared UI in `src/components/ui/`.
- **Components:** One component per file. Co-locate component-specific types in the same file. Shared types go in `src/types/`.
- **API responses:** All API responses use `{ data, error, meta }` shape via tRPC
- **Database:** All queries go through Prisma. No raw SQL unless explicitly approved. Use transactions for multi-table operations.
- **Testing:** Every new feature needs unit tests for business logic and at least one e2e test for the primary user flow.
- **Imports:** Use `@/` path alias for `src/` directory. Group imports: external → internal → relative.

Boundaries

Always

- Run `pnpm test` before marking any task complete
- Follow existing file and folder structure in `src/features/`
- Use existing UI components from `src/components/ui/` before creating new ones
- Use Prisma for all database operations
- Use tRPC for all API endpoints (no raw API routes for new features)
- Handle loading and error states in all UI components
- Use TypeScript strict mode — no `any` types, no `@ts-ignore`

Ask

- Before adding any new `pnpm` dependency
- Before creating a new top-level directory under `src/`
- Before modifying `prisma/schema.prisma` (requires migration planning)
- Before changing any authentication or authorization logic
- Before modifying shared UI components in `src/components/ui/`
- Before changing the tRPC router structure

Never

- Never commit `.env` files or expose API keys/secrets
- Never push directly to `main` — all changes go through PRs
- Never skip or disable tests
- Never modify Vercel or Railway configuration files
- Never use `!important` in Tailwind/CSS
- Never bypass NextAuth for authentication checks

- Never use `console.log` in production code — use the logger from `src/lib/logger.ts`